

Data Cleaning & Transformation

1. Raw Data Preparation

- Imported the original Preply earnings data into Power Query.
- Reviewed column names, data types, missing values and record structure.
- Standardised date, text and numerical fields.
- Key source columns: Student, Lesson Date, Type (Trial / regular), Lesson Price,USD, Earning,USD.

2. Lesson Classification

- Standardised lesson records into Trial and Non-trial categories using the Type column, filtering to Trial records for acquisition analysis (one trial per student, by Preply's own structure).
- Prepared the lesson-level data required to distinguish initial student acquisition from subsequent lessons.

3. Effective Price 2

The raw Lesson Price,USD column could not be used directly for pricing analysis. Some trial lessons were booked at 25 minutes (half price) rather than the standard 50 minutes, which meant raw price alone did not reliably represent the tutor's actual pricing decision at a given point in time. Effective Price 2 was built through a four-step correction pipeline to resolve this before it could be used as the primary pricing variable.

Step 1 — Decimal Corrected: any price with a decimal value is guaranteed to be a 25-minute lesson (half-length lessons never round to a whole number), so these are doubled to their full-lesson equivalent.

Decimal Corrected =

if Number.Mod([Lesson Price,USD], 1) <> 0

then [Lesson Price,USD] * 2

else [Lesson Price,USD]

Step 2 — Added Index: a row index is added (Add Column → Index Column → From 0) to support the row-above lookup used in the next step.

Step 3 — Effective Price 1: catches the remaining ambiguous whole-number half-price cases (e.g. \$6 when the full price was \$12) by checking whether a price is exactly half of the row directly above it.

Effective Price 1 =

if [Decimal Corrected] = (try #"Added Index"{{[Index]-1}[Decimal Corrected] otherwise null) / 2

then (try #"Added Index"{{[Index]-1}[Decimal Corrected] otherwise null)

else [Decimal Corrected]

Known-impossible values (below the tutor's historical price floor of \$5) were also hardcoded, since these could only exist as half-price artifacts: 3 → 6, 4 → 8.

Step 4 — Effective Price 2: remaining ambiguous cases (where two prices coincidentally shared a valid half/full relationship) were manually verified against Google Calendar lesson durations, then corrected student-by-student.

Effective Price 2 =

if [Student] = "Name1" then 10

else if [Student] = "Name2" then 12

else [Effective Price 1]

This field became the primary pricing variable used to analyse changes in student acquisition, returning students and earnings across different price points.

4. Earnings Calculations

Gross and net pay were derived directly from the two raw earnings columns, no transformation was needed since trial lessons already pay \$0 by Preply's own structure.

Gross Pay = SUM('Earnings Fact Table'[Lesson Price,USD])

Net Pay = SUM('Earnings Fact Table'[Earning,USD])

Monthly Earnings uses the same Net Pay logic, aggregated by the DimDate month-level fields described in Section 5.

5. Date & Monthly Preparation

A Price Period (month-start) column was added in Power Query to collapse individual lesson dates into a consistent monthly grouping for the acquisition and revenue analyses, while still preserving each distinct price within that month (see Section 6).

Price Period = Date.StartOfMonth([Lesson Date])

Rows with a blank Lesson Date (representing the tutor's Preply signup record rather than an actual lesson) were excluded from all lesson-based visuals via a "Lesson Date is not blank" filter, since DimDate naturally does not generate a row for them.

6. Student-Level Calculations

Month + Price grouping (acquisition table): the trial-level table was grouped by Price Period and Effective Price 2 together, so that multiple distinct prices within the same month remain visible as separate rows rather than being collapsed into a misleading monthly average.

Returned flag (student-level): a helper table was built by grouping the full Earnings Fact Table by Student and counting total rows per student, then flagging students with more than one row as returned.

Returned = [Booking Count] > 1

Returned Count Helper = if [Returned] then 1 else 0

This flag was merged back into the trial-level table (matched on Student), then grouped by Price Period + Effective Price 2 a second time, with Total Students (count of students) and Returned Students (sum of Returned Count Helper) as the two aggregations, producing the acquisition-vs-return table used throughout the pricing analysis.

Return Rate % = [Returned Students] / [Total Students] * 100

7. Power BI / DAX Measures

The cleaned data was loaded into Power BI, where DAX measures were created for the final analysis.

Trial Students

Trial Students = CALCULATE(DISTINCTCOUNT('Earnings Fact Table'[Student]), 'Earnings Fact Table'[Type] = "Trial")

Returned Students

Returned Students =

```
COUNTROWS(
    FILTER(
        VALUES('Earnings Fact Table'[Student]),
        CALCULATE(
            COUNTROWS('Earnings Fact Table')
        ) > 1
    )
)
```

This evaluates each distinct student (VALUES), then, for each one, uses CALCULATE to convert that row context into real filter context on the fact table, counting how many rows that specific student has. Students with more than one row (i.e. more than just their trial) are counted as returned.

Return Rate

Return Rate = DIVIDE([Returned Students], DISTINCTCOUNT('Earnings Fact Table'[Student]))

Gross Pay

Gross Pay = SUM('Earnings Fact Table'[Lesson Price,USD])

Net Pay

Net Pay = SUM('Earnings Fact Table'[Earning,USD])

Other Dashboard Measures

Monthly Lesson Count = COUNT('Earnings Fact Table'[Lesson Date])

Average Effective Price = AVERAGE('Earnings Fact Table'[Effective Price 2])

Median Effective Price = MEDIAN('Earnings Fact Table'[Effective Price 2])

Median Effective Price was used in place of a simple average in views prone to distortion by outlier prices (e.g. package/bundle prices sitting far outside the normal \$5–\$25 trial range), since median is not pulled off-center by a small number of extreme values the way a mean is.

8. Date Model

A dedicated DimDate table was created to support consistent time-based analysis, generated dynamically off the actual minimum and maximum lesson dates in the data rather than a hardcoded range, so it automatically adjusts on refresh.

DimDate =

```
CALENDAR(
    DATE(
        YEAR(MIN('Earnings Fact Table'[Lesson Date])),
        MONTH(MIN('Earnings Fact Table'[Lesson Date])),
        1
    ),
    MAX('Earnings Fact Table'[Lesson Date])
)
```

)

9. Data Model & Relationship Notes

- The model includes a Monthly Frequency Tier table (many) → Engagement table (one) → Earnings Fact Table (many), supporting secondary engagement analysis outside the core pricing model.
- The Monthly Frequency Tier → Engagement relationship required bidirectional cross-filtering, since by default a filter on the "many" side (Frequency Tier) could not propagate to the "one" side (Engagement) to correctly filter average engagement span by tier.
- A duplicate-student-name issue was identified and confirmed as genuinely different students sharing a name (not a data error), since a single student can only have one trial by definition. This remains an open item for building a fully reliable cross-table relationship key, and the pricing/acquisition analysis in this project does not depend on that relationship.
- An earlier "price era" methodology (incrementing a counter each time price changed row-over-row) was built and tested, but was ultimately replaced by the simpler Month + Effective Price 2 grouping described above, after discovering that lesson date does not always equal the date a student actually selected a given price, which undermined the precision the era approach was designed to provide.